

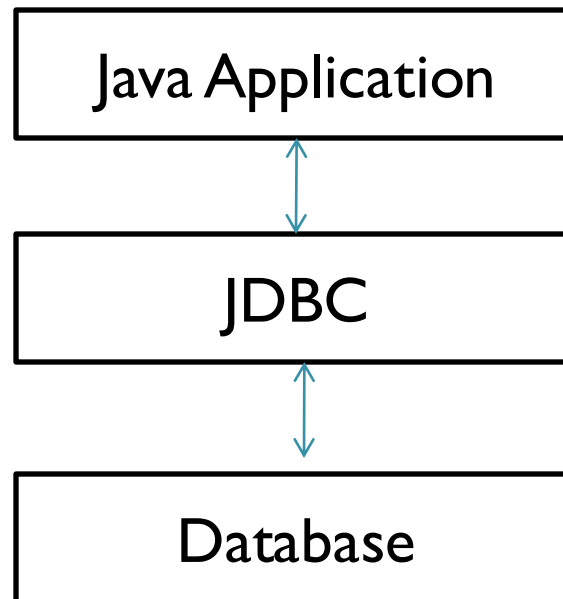
Java Database Connectivity (JDBC)

Refer :

<https://github.com/sagaruppuluri/EP/blob/main/Module4/Readme.md>

JDBC API

JDBC API provides the means to execute SQL statements within the java program.



Major steps involved

- 1) Import necessary classes
- 2) Register Driver
- 3) Establish Connection
- 4) Create Statement object
- 5) Execute database specific commands
- 6) If command is a query, then obtain ResultSet
- 7) Process ResultSet
- 8) Close Statement
- 9) Close Connection.

Establishing Connection

JDBC database drivers implement **Driver** interface.

DriverManager is responsible for establishing connections through JDBC drivers. To establish connection

1) Register driver :-

To register driver use,

```
DriverManager.registerDriver( driverClassInstance );
```

All JDBC drivers come up with *static initializer* which will register themselves under DriverManager. So, it is sufficient to load the driver class. Use,

```
Class.forName("driverclass");
```

e.g.

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

Establishing Connection(Continued)

2) Connecting to the database :-

Use DriverManager.getConnection(...) to connect to the database.

Connection con =

DriverManager.getConnection(url , user , password);

url :- Format of the JDBC url is

jdbc : subprotocol : subname

The content of subname depends on subprotocol.

e.g.

jdbc : mysql : // localhost : 3306 / DemoDB

user, password :- Database specific.

Creating the Statement object

Statement object is used to execute database specific commands such as insert , delete etc. It must be created using `createStatement( )` method on a valid `Connection` object.

```
Statement st = con.createStatement( );
```

Use it to execute SQL commands; as,

```
boolean result = st.execute("create table.....");
```

```
int n = st.executeUpdate("insert .....");
```

```
ResultSet rs = st.executeQuery("select * from dept");
```

Processing the ResultSet

When the statement is a query then it will result in a ResultSet object.

e.g. `ResultSet rs = st.executeQuery("select * from dept");`

Basic view of the ResultSet is

Initial pos	⇒	BOF		
After next()	⇒	10	Accounting	New York
		20	Research	Dallas
		EOF		

Use the `getXXX(..)` methods to retrieve the field values.

e.g. `rs.getInt(1)` (or) `rs.getInt("deptno")`
`rs.getString(2)` (or) `rs.getString("dname")`
`rs.getString(3)` (or) `rs.getString("loc")`

Execute methods of the Statement Object

ResultSet executeQuery (query) :-

If the command is a query then executeQuery should be used.

e.g.

```
ResultSet rs = st.executeQuery("select * from dept");
```

int executeUpdate (dmlCommand) :-

If the command is a DML command returning update count then executeUpdate should be used.

e.g.

```
int n = st.executeUpdate(" delete from temp " );
```

Return value represents the no-of records affected by the command. i.e **n records deleted.**

Execute methods of the Statement Object

boolean execute(command) :-

If you are not sure of the type of command that is used then use execute. This method **returns true** if the command produces a ResultSet and **false** if it returns update count.

e.g.

1) `st.execute("create table.....");`

```
2) boolean flag = st.execute( someCommand );
   if( flag == true ) {
       ResultSet rs = st .getResultSet( );
       .....
   }
   else {
       int n = rs.getUpdateCount ( );
       .....
   }
```

Batch Updates

Instead of executing each SQL command individually, you can prepare a batch and execute them all at once. It reduces the no-of round trips to the database.

e.g.

```
Statement st = con.createStatement( );
st.addBatch( "insert into ..... " );
st.addBatch( " update..... " );
st.addBatch( " delete ..... " );
int updateCounts [ ] = st.executeBatch( );
System.out.println(
    " no of records inserted " + updateCounts[0] );
System.out.println(
    " no of records updated " + updateCount[1] );
```

Note: you should not add select command to the batch.

Statement Types

- ❑ Statement
- ❑ PreparedStatement – For parameterized commands.
- ❑ CallableStatement – For stored procedure calls.

Statement :-

Use createStatement to create the Statement object;
as,

```
Connection con = .....;  
Statement st1 = con.createStatement( );  
Statement st2 = con.createStatement( );
```

Use two different Statement instances to obtain two ResultSet instances at a time (or) use join.

PreparedStatement

PreparedStatement extends Statement by adding the ability to set values for parameter markers . Use **prepareStatement** method to create an instance of type PreparedStatement.

e.g.

```
Connection con = .....;
PreparedStatement ps =
    con.prepareStatement (
        “ insert into dept values ( ? , ? , ? ) ” );
```

```
ps.setInt( 1 , 99 );
ps.setString( 2 , “abc” );
ps.setString( 3 , “Guntur” );
```

```
int n = ps.executeUpdate( );
```

Parameter markers



Parameter number
followed by value



CallableStatement

The CallableStatement interface extends PreparedStatement with methods for executing and retrieving results from stored procedures. Use **prepareCall(..)** to create an instance of type CallableStatement.

Create the following procedure :-

```
create or replace procedure
ins( no in number , dname in varchar2 , loc in varchar2 )
is
begin
    insert into dept values( no , dname , loc );
end;
/
```

CallableStatement (continued)

```
Connection con = .....
```

```
CallableStatement cs =  
    con.prepareCall( " { call ins( ? , ? , ? ) } " );
```

```
cs.setInt( 1 , 98 );  
cs.setString( 2 , "abc" );  
cs.setString( 3 , "Guntur" );
```

```
cs.execute ( );
```

Transaction control

A transaction is a set of one or more SQL statements that make up a logical unit of work. A transaction ends with either a commit or a rollback. By default the **auto-commit** option is set to **true** in the **Connection** object. It will automatically commit each individual SQL statement as soon as it is complete. If you need to manually commit or rollback a transaction first disable auto-commit by setting it to false.
i.e.

```
con.setAutoCommit( false );  
..... updates  
if( successful )  
    con.commit( );  
else  
    con.rollback( );
```

Transaction control (continued)

Note:-

If auto-commit feature is disabled you have to manually commit or rollback a transaction before closing the connection. Otherwise it leads to **InvalidTransactionStateException**.

Transaction Isolation Levels

Transaction isolation levels specify what data is “visible” to the statements within a transaction. Possible interactions between concurrent transactions is categorized as follows

Dirty reads :-

Occur when a transaction is *allowed to read uncommitted data*. What happens if the changes are rolled back by the other transaction?

Non-repeatable reads :-

- a. Transaction A reads a row
- b. Transaction B changes the row
- c. Transaction A reads the same row again and gets different results.

Transaction Isolation Levels(Continued)

Phantom reads :-

- a. Transaction A reads all rows that satisfy some **where** condition.
- b. Transaction B inserts a row that satisfies the same condition.
- c. Transaction A reevaluates the **where** condition and picks up the additional “**phantom**” row.

Transaction Isolation Levels(Continued)

Transaction isolation levels which can be set are

- 1) **TRANSACTION_NONE** :- Indicates driver does not support transactions. i.e. not a JDBC complaint driver.
- 2) **TRANSACTION_READ_UNCOMMITTED** :- Allows transactions to see uncommitted data. Dirty reads, non-repeatable reads and phantom reads are possible.
- 3) **TRANSACTION_READ_COMMITTED** :- Allows transactions to see only committed data. Dirty reads are not possible , but non-repeatable reads and phantom reads are possible.
- 4) **TRANSACTION_REPEATABLE_READ** :- Disallows dirty reads and non-repeatable reads, but phantom reads are still possible.
- 5) **TRANSACTION_SERIALIZABLE** :- Specifies dirty reads, non-repeatable reads and phantom reads are prevented.

Transaction Isolation Levels(Continued)

The **Connection** method **setTransactionIsolation** is provided to allow JDBC clients to change the transaction isolation level for a given Connection object.

```
Connection con = ..... ;  
con.setTransactionIsolation(  
    Connection.TRANSACTION_SERIALIZABLE );
```

Note:

Driver may or may not support the specified transaction isolation level, verify.